

16 - Prática: Redes Neurais Convolucionais 2 (Deep Learning) (II)

Thassiana C. A. Muller

Convolutional Neural Networks

Introdução

Os fundamentos de CNNs tais como: o que é convolução, *kernel*, mapa de características, *pooling* e *flattening*, gradiente, *mean square error* foram detalhados nos relatórios dos cards [15 - Prática_Redes Neurais Convolucionais 1 \(Deep Learning\) \(II\).pdf](#) e [13 - Redes Neurais](#)

O objetivo deste relatório é aprofundar os conhecimentos adquiridos anteriormente e aprender novas perspectivas na área de estudo de CNNs.

Convolução - Uma nova perspectiva

A Convolução pode ser vista como uma forma de **encontrar padrões** em um conjunto de dados, normalmente, mas não somente, em imagens.

Vetorização

O curso traz uma abordagem de vetorização de operações de código para melhor integração com as funções do *NumPy*.

Produto Escalar

Definido como:

$$a \cdot b = \sum_{i=1}^N a_i b_i = |a||b| \cos \theta_{ab}$$

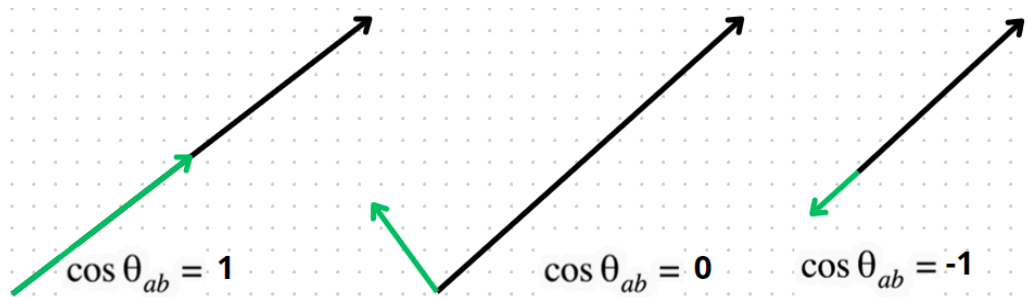
É uma operação entre dois vetores que resulta em um número real. Ele mede o quanto dois vetores "apontam" na mesma direção. Dessa forma, expressa a correlação entre dois pontos, **pontos muito próximos terão um produto escalar alto e positivo e pontos opostos terão um produto escalar mais alto e negativo**. Caso não haja correlação entre os pontos, seu produto escalar é zero (ortogonais).

Similaridade do cosseno

Definida como:

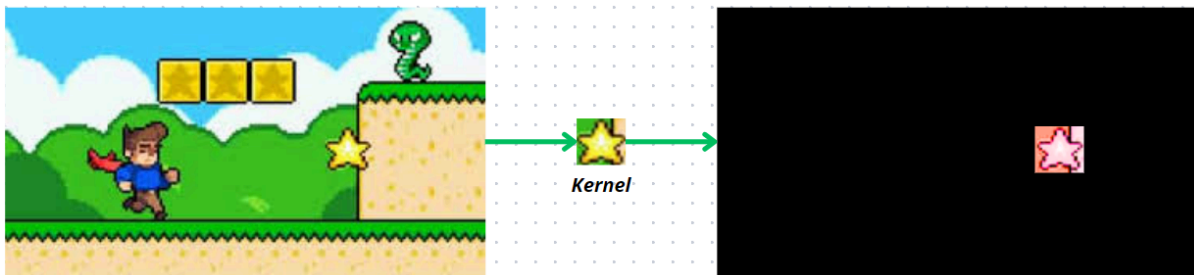
$$\cos \theta_{ab} = \frac{a \cdot b}{|a||b|}$$

Expressa a relação entre dois vetores *a* e *b*. Seu valor varia entre -1 e 1, quanto mais próximo de 1 mais similares são os vetores.



Convolução e filtros

No card anterior sobre esse mesmo tema, vi que o filtro ou *kernel* era apresentado como uma matriz que deslizava sobre a matriz a ser convolucionada, aplicando operações matemáticas que modificavam a matriz original para destacar suas características mais relevantes. Já no curso atual, da pra imaginar o conceito de filtro de uma maneira mais simples: ele também desliza sobre a matriz, mas com o objetivo de extrair a informação de localização de um conjunto específico de características. Ou seja, em vez de transformar a matriz inteira, ele **filtra o que está sendo procurado**.



fonte: adaptação de Super Steve World

Translational invariance (CNN) vs fully connected (ANN clássica)

CNNs

- Nas CNNs, os **filtros convolucionais deslizam** sobre a imagem.
- Isso significa que **o mesmo padrão pode ser detectado em qualquer posição**, porque o filtro é aplicado em toda a imagem.
- Exemplo: se um olho está no topo ou no canto da imagem, a CNN ainda pode reconhecer que é um olho.
- Essa capacidade de "não ligar para a posição exata" é chamada de **invariância a translações**.
- Além disso, **camadas de pooling** ajudam a reforçar essa invariância, resumindo regiões da imagem e mantendo a informação mais geral.

ANNs clássicas - fully connected

- Numa rede neural clássica (fully connected), **cada pixel da imagem entra em um neurônio específico.**
- Se o objeto se move um pouco (ou seja, é transladado na imagem), os valores dos pixels mudam de posição, e isso muda completamente a entrada da rede. Sendo necessário o **retreinamento dos pesos para o reconhecimento do mesmo objeto em posições diferentes**
- Resultado: a rede não reconhece mais o padrão, a **posição absoluta importa muito.**

Convolução discreta 1D

Definida como:

$$\sum_{i'=0}^{K-1} a_{i+i'} \cdot w_{i'}$$

Onde:

- a: vetor de entrada (por exemplo, uma linha de pixels ou sequência de dados).
- w: vetor de pesos (ou kernel, ou filtro), com comprimento K.
- i': índice que percorre os elementos do filtro, de 0 até K-1
- i: posição atual onde o filtro está “centralizado” ou “alinhado” com a entrada.

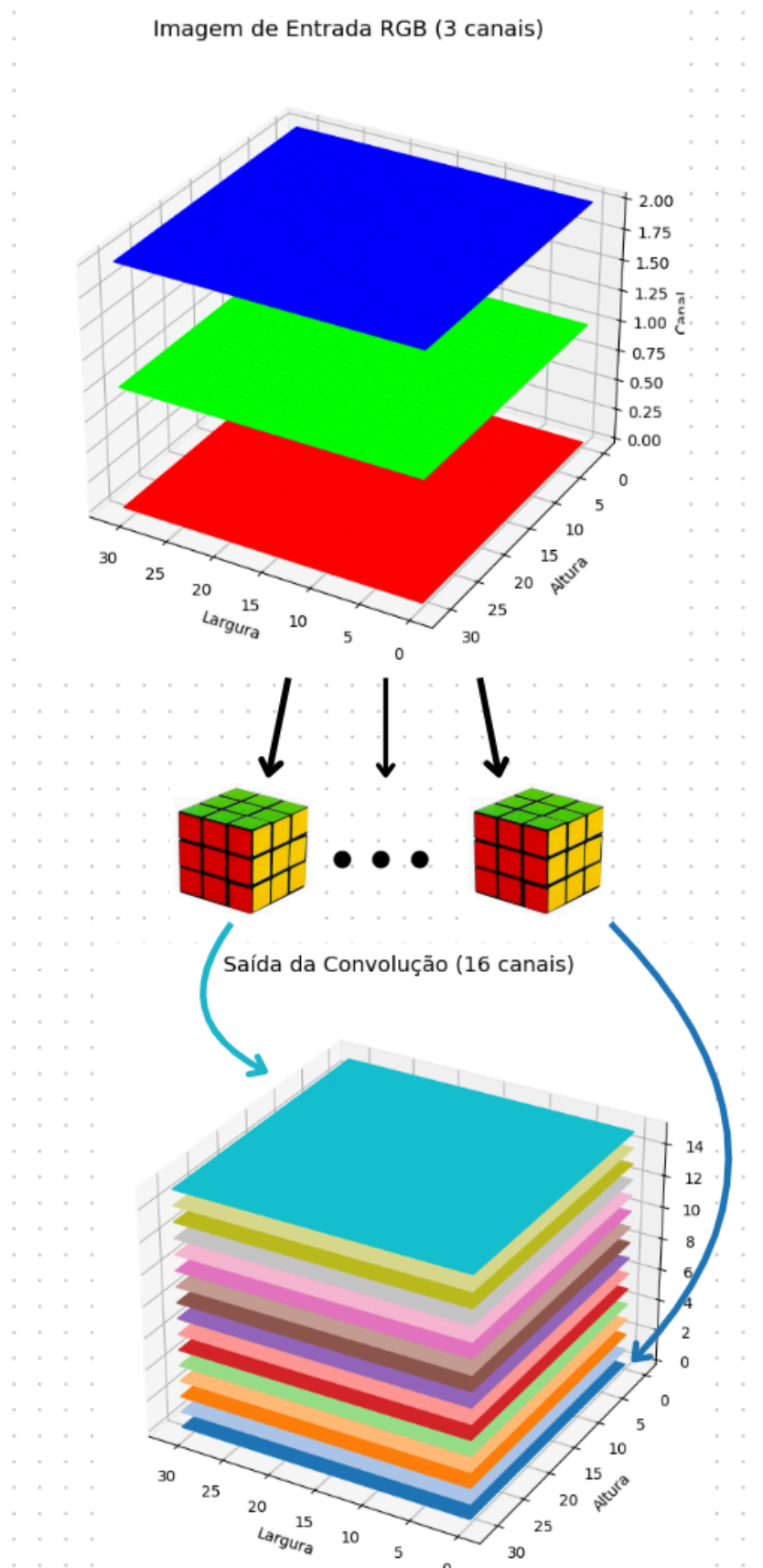
Dessa forma, é uma **soma ponderada de valores locais da entrada**, usada para capturar padrões, como variações rápidas (bordas) ou tendências locais. Ao mover esse cálculo ao longo da entrada, é gerado uma nova versão da entrada, destacando certas características.

Convolução em imagens coloridas - 3D convolution

Imagens coloridas possuem 3 dimensões, altura x largura x cor. A ideia é parecida com a convolução de 2 dimensões, mas ao invés de um filtro “quadrado” deslizar sobre a entrada, um filtro “cúbico” fará isso.

Ao aplicar a **convolução em uma imagem 3D o mapa de características resultante seria 2D** pois o filtro desliza em nas dimensões “x” e “y” mas não desliza na profundidade (a profundidade do kernel é a mesma da entrada). Assim, para aplicar a próxima convolução, convém utilizar a extração de múltiplas características por meio de vários kernels de forma a obter uma saída “altura x largura x n° de kernels”.

Exemplo de convolução em uma imagem colorida com 16 kernels (representados por cubos mágicos) diferentes



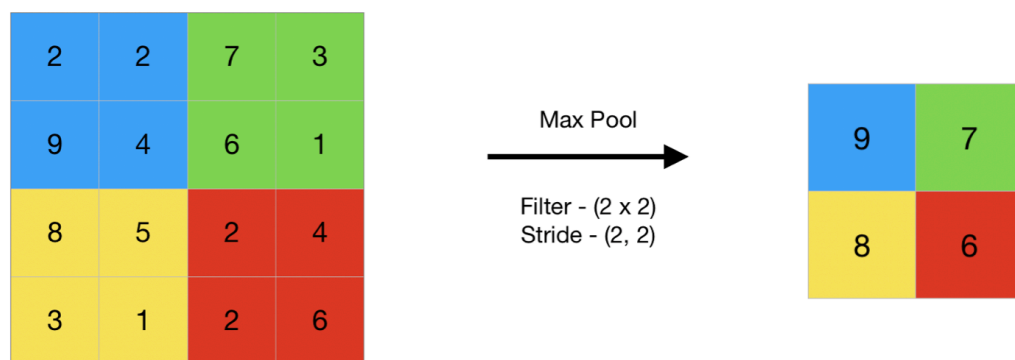
Fonte: Autoria própria

Bias em entradas de n dimensões

O bias de convoluções de dimensões $A \times L \times P$ será um vetor de tamanho P , o numpy se encarrega de aplicar corretamente esse bias a cada mapa de ativação por meio de broadcasting.

Pooling

Usado para extrair características locais o pooling normalmente sucede as convoluções, pois isso permite à rede generalizar as informações, extraindo apenas as características mais relevantes. Pixels próximos tendem a ter valores semelhantes, o que torna desnecessário manter todos os detalhes. Além disso, o pooling reduz a dimensionalidade dos dados, tornando o processamento mais eficiente nas etapas seguintes da rede neural. Por exemplo, nas primeiras camadas a rede aprende a detectar linhas e bordas; nas camadas intermediárias, começa a identificar partes do rosto; e nas últimas, consegue reconhecer o rosto como um todo.

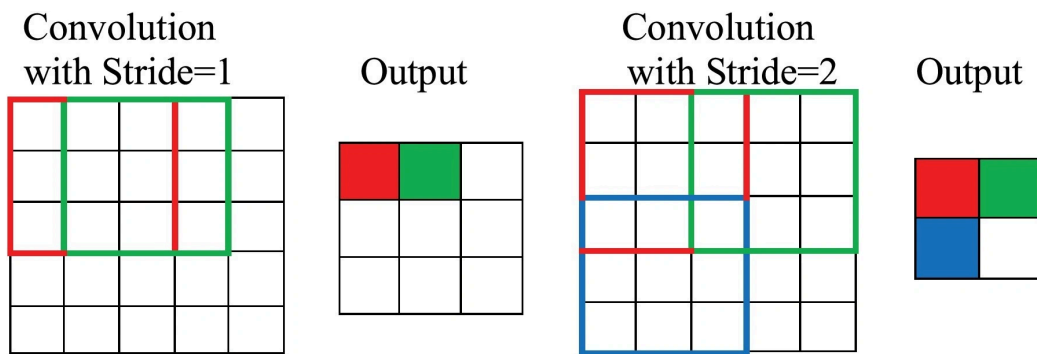


Fonte: [geeksforgeeks.org](https://www.geeksforgeeks.org/)

Stride

Representa o passo com que a janela se desloca sobre a imagem ou feature map. Por exemplo, em uma operação de max pooling com janela 2x2 e stride 2, a janela se move dois pixels por vez, reduzindo pela metade as dimensões da entrada. Esse parâmetro ajuda a tornar a rede mais eficiente e menos sensível a pequenas variações na posição dos elementos da imagem. Porém, caso o stride seja menor que o tamanho da janela, pode ocorrer sobreposição.

Em algumas CNNs, o **pooling pode ser substituído por convoluções com stride maior**, como stride 2. Isso porque uma convolução com stride > 1 não só extrai características, mas também **reduz a dimensionalidade** da saída, funcionando como uma forma de *downsampling*. A vantagem é que convoluções com stride aprendem pesos, o que pode tornar o modelo mais expressivo e eficiente, enquanto o pooling é uma operação fixa e não-paramétrica. Por isso, muitas redes recentes como ResNet e GoogleNet usam essa substituição.



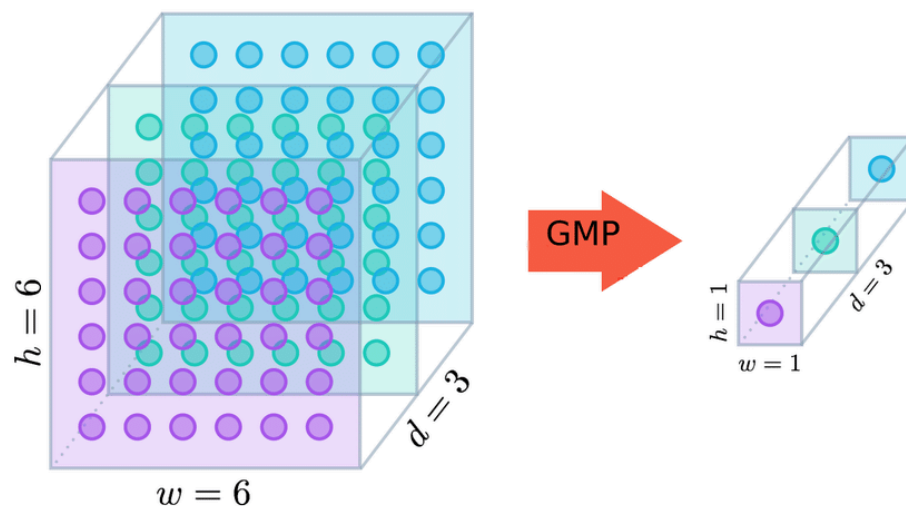
Fonte: Computer.org

Perda de informação durante pooling ou convolução com stride > 1?

No contexto de CNNs o que importa é a invariância translacional, ou seja, **não nos interessa saber necessariamente em qual posição da imagem original tal informação estava, mas sim que a informação estava presente**. Dessa forma, mesmo com a perda de dados espaciais causada pela redução do tamanho da entrada ao longo das convoluções, informações relevantes de quais *features* foram encontradas são mantidas

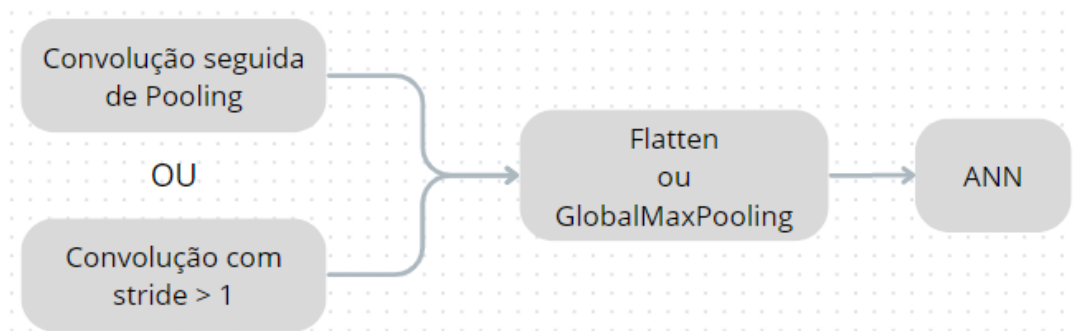
Global Max Pooling

Reduz cada mapa de ativação a um único valor, selecionando o maior valor presente em toda a matriz, assim, considera toda a dimensão espacial de uma vez, **transformando cada feature map em um único número**. Mantém apenas a informação mais dominante de cada mapa de ativação.



Fonte: [Himanshu Singhal](#)

Dessa forma temos algo como:



Fonte: Autoria própria

Data augmentation

Usa a lógica de generators para **aumentar a diversidade dos dados de treinamento** sem consumir muita memória, já que as transformações são aplicadas em tempo real durante o treinamento através do yield. Ao invés de pré-processar e armazenar todas as variações das imagens, o generator cria novas imagens a cada vez, aplicando transformações como rotações, inversões e ajustes de brilho.

Batch Normalization

Normalizando as ativações de cada camada fazendo com que os valores tenham média próxima de zero e variância próxima de um dentro de cada mini-batch, **reduz o *internal covariate shift***, que é a variação na distribuição das entradas de uma camada causada pela atualização dos pesos das camadas anteriores. Além de estabilizar o treinamento, a batch normalization permite o uso de taxas de aprendizado maiores e ajuda a evitar overfitting.

Natural Language Processing (NLP)

Introdução

O tópico aborda o porquê de utilizar vetores para representar palavras devido a localização espacial de palavras de contextos similares ficarem próximas espacialmente. Também aprofunda conceitos de convoluções aplicadas no processamento da linguagem natural.

Word Embeddings

Consiste em representações vetoriais de palavras ou frases em um espaço multidimensional, onde a proximidade entre vetores reflete a similaridade semântica entre os termos.

Um exemplo disso é palavra “Endereço” e “Local” que utilizando word embeddings podem ser representadas pelo vetores abaixo:

$$\mathbf{v}_{\text{Endereço}} = [0.25, -0.12, 0.78, \dots] \quad \mathbf{v}_{\text{Loc}} = [0.30, -0.10, 0.80, \dots]$$

CNNs para NLP e sequências

Parecido com convoluções em imagens, porém a convolução será 1D. A ideia continua a mesma, filtrar padrões em dados. Agora a primeira camada que será convolucionada são os embeddings textuais.

Funções de perda

Binary Cross Entropy

É a **função de perda** usada quando o modelo faz uma **tarefa de classificação binária**, ou seja, quando há apenas duas classes possíveis. Ela mede a diferença entre a probabilidade prevista pelo modelo (valores entre 0 e 1 após uma função sigmoide) e os rótulos reais (0 ou 1). A fórmula penaliza previsões com alta confiança e erradas, incentivando o modelo a ser preciso. Quanto menor o valor da entropia cruzada, melhor o desempenho do modelo em separar corretamente as duas classes.

Categorical Cross Entropy

Usada em tarefas de classificação multiclasse, onde a saída esperada é um vetor one-hot com a classe correta marcada como 1 e o restante como 0. Essa função compara a distribuição de probabilidade prevista (obtida geralmente com uma softmax) com a distribuição real e calcula a perda com base na diferença entre elas. Assim como a binary cross entropy, ela **penaliza mais fortemente quando o modelo erra com alta certeza, mas lida com múltiplas classes**.

Gradient Descent

Stochastic Gradient Descent (SGD)

É uma versão simplificada do método de descida do gradiente em que os pesos são atualizados usando apenas um exemplo (ou um pequeno lote) por vez. Isso torna o **treinamento mais rápido e menos custoso computacionalmente** em grandes conjuntos de dados. Embora SGD possa ser ruidoso, esse ruído pode ajudar a escapar de mínimos locais e melhorar a generalização, principalmente quando combinado com outras técnicas como momentum ou taxa de aprendizado adaptativa.

Momentum

Serve para acelerar o SGD acumulando uma média exponencial dos gradientes passados e usando essa média para atualizar os pesos. Isso reduz a oscilação em direções de menor importância e acelera em direções mais consistentes, funcionando como uma forma de inércia. Em termos práticos, o momentum **ajuda o otimizador a manter a direção certa e alcançar o mínimo global mais rapidamente**, especialmente em áreas com curvas rasas ou vales estreitos na superfície de erro.

Variable and Adaptive Learning Rates

Técnicas que ajustam dinamicamente a taxa de aprendizado durante o treinamento. Em vez de usar uma taxa única, pode ser utilizado estratégias como *learning rate schedules* que reduzem o valor com o tempo ou otimizadores adaptativos que permitem ajustes com base no progresso do treinamento. Isso ajuda a estabilizar o aprendizado pois **passos maiores no início aceleram a convergência, e passos menores no final refinam os pesos com mais precisão**.

Adam (adaptive moment estimation)

Combina os benefícios do momentum com taxas de aprendizado adaptativas. Ele mantém duas médias móveis: uma dos gradientes (como o momentum) e outra dos quadrados dos gradientes, permitindo ajustes mais refinados nos pesos. Essas estimativas são corrigidas para compensar o viés inicial e resultam em uma atualização eficiente, especialmente útil em problemas com muitos parâmetros ou gradientes esparsos.

